

Good. Now I'll give you the **final, strict file structure** exactly aligned with your constraints:

- ☒ Full OOPS
- ☒ Only .cpp files
- ☒ No header files
- ☒ README updated after every step
- ☒ /doc folder must always be referenced
- ☒ Saved weights file allowed
- ☒ Terminal-based

This is the **clean, final architecture blueprint**.

FINAL PROJECT STRUCTURE

SoundClassifierNN/

```
|
|
|— Cmake.txt
|— main.cpp
|
|— AudioLoader.cpp
|— FeatureExtractor.cpp
|— Preprocessor.cpp
|— NeuralNetwork.cpp
|— Trainer.cpp
|— Evaluator.cpp
|— Utils.cpp
|
|— README.md
|
|— data/
| |— clap/
| | |— clap_01.wav
| | |— clap_02.wav
```

```
| | └─ ...
| |
| └─ knock/
| └─ noise/
| └─ silence/
|
| └─ models/
| └─ trained_weights.dat
| └─ normalization_params.dat
|
| └─ results/
| └─ training_log.txt
| └─ confusion_matrix.txt
| └─ metrics.txt
|
└─ doc/
```

FILE RESPONSIBILITY BREAKDOWN (STRICT OOPS)

◆ **main.cpp**

Role: System Orchestrator

Responsibilities:

- Select training or inference mode
- Call modules in correct sequence
- Never contain math logic
- Never contain feature extraction code
- Never contain neural network math

Main should only coordinate objects.

◆ **AudioLoader.cpp**

Class: AudioLoader

Responsibilities:

- Read WAV file
- Validate 16kHz
- Convert PCM → double
- Return vector

No feature logic.

No NN logic.

◆ FeatureExtractor.cpp

Class: FeatureExtractor

Responsibilities:

- Divide into 20 frames
- Compute 6 features:
 - Total energy
 - Mean amplitude
 - Variance
 - ZCR
 - Frame energy mean
 - Frame energy variance

Stateless class.

No training code.

◆ Preprocessor.cpp

Class: Preprocessor

Responsibilities:

- Fit normalization on training set only
- Store mean & std
- Transform features
- Save normalization parameters
- Load normalization parameters

Must persist parameters to:

models/normalization_params.dat

◆ **NeuralNetwork.cpp**

Class: NeuralNetwork

Responsibilities:

- Weight initialization (He)
- Forward propagation
- Backpropagation
- Gradient descent update
- Save weights
- Load weights

Must persist weights to:

models/trained_weights.dat

Must NOT:

- Load audio
 - Extract features
 - Split dataset
-

◆ **Trainer.cpp**

Class: Trainer

Responsibilities:

- Shuffle training data
- Loop epochs
- Call forward/backward
- Track loss
- Track training accuracy
- Track validation accuracy
- Write logs to:

results/training_log.txt

No feature extraction here.

◆ Evaluator.cpp

Class: Evaluator

Responsibilities:

- Run inference on test set
- Compute accuracy
- Compute confusion matrix
- Save metrics to:

results/metrics.txt

results/confusion_matrix.txt

◆ Utils.cpp

Class: Utils

Responsibilities:

- Matrix multiplication helpers
- Random number generator
- One-hot encoding
- File reading helpers
- Argmax function

Must remain utility-only.

DATA FLOW STRUCTURE

Training Mode

Load dataset paths

↓

AudioLoader

↓

FeatureExtractor

↓

Preprocessor.fit()

↓

Preprocessor.transform()

↓

Trainer.train()

↓

NeuralNetwork.saveModel()

↓

Preprocessor.saveParams()

Inference Mode

NeuralNetwork.loadModel()

Preprocessor.loadParams()

↓

AudioLoader

↓

FeatureExtractor

↓

Preprocessor.transform()

↓

NeuralNetwork.forward()

↓

Prediction

README.md RULES

After EVERY development step:

Append:

Step X Completed

- Modules implemented:

- Methods added:

- Files modified:

- How to test:

- Known issues:

README must act as a progress log.

DOC FOLDER RULE

Before implementing any module:

1. Open corresponding Word file in /doc
 2. Follow mathematical definitions exactly
 3. Do not improvise architecture
 4. If conflict arises → doc file wins
-

STRICT NO HEADER POLICY

All classes must:

- Be fully defined in their .cpp file
 - Contain class declaration and implementation in same file
 - Use forward declarations where needed
 - Avoid circular dependencies
-

COMPILATION STRUCTURE

Compile using:

```
g++ *.cpp -o SoundClassifierNN
```

No additional flags unless necessary.

ARCHITECTURAL TRADE-OFF YOU'RE MAKING

By removing header files:

You gain:

- Simpler academic structure
- Less file management overhead

You lose:

- Clear interface separation
- Faster incremental compilation
- Large-scale scalability
- Clean multi-developer structure

For college project → acceptable.
For industry system → not ideal.